

THE UNLOX ACADEMY

Live Project · Weeks 1+2 · 2-Hour Industry Sprint

Bangalore Tech Salary Decoder

*Build a salary-intelligence report on 1,000 Bengaluru tech professionals – in 2 hours.***INDUSTRY CONNECT****The 2-hour LinkedIn challenge.**

Every student in this room will apply to 50+ jobs in the next six months. Every one of them is currently guessing what CTC to ask for. Nobody - not your college TPO, not LinkedIn's free tools, not the 'CTC dump' WhatsApp groups - gives you a quantitative answer broken down by role, experience, company type, skills, and education tier.

In the next 2 hours, you are going to build that tool. You will receive a dataset of 1,000 Bengaluru tech professionals. You will clean it, analyse it, and produce a printed report that any recruiter, peer, or junior would screenshot.

By 5 PM today, your post will go up on LinkedIn: "I analysed 1,000 Bangalore tech salaries in 2 hours. Here's what I found." That post will get you noticed.

Time: 2 hours, single sitting · **Dataset:** bangalore_tech_salaries.csv

Deliverable: one .ipynb · Section-wise commented · Final printed report

The Dataset

You will receive a single CSV file: `bangalore_tech_salaries.csv`. It contains 1,015 rows representing tech professionals working in Bengaluru as of 2024. The data is synthetic - generated by us - but designed around realistic Indian tech compensation patterns. The dataset is intentionally messy to mirror what real HR data looks like.

Columns

- `Employee_ID` — unique identifier (BLR0001 to BLR1000)
- `Role` — job title (has variants like 'SDE Backend', 'Backend Engineer', 'Backend Dev', 'BE' that all mean the same role)
- `years_exp` — total years of professional experience (integer)
- `Current_CTC` — current cost-to-company. STORED AS STRING in 4 different formats: '15.5 LPA', '15.5', '15,50,000', '₹15.5 LPA'
- `Previous_CTC` — last drawn CTC. Same format mess. NULL for freshers and career changers (~20% of rows).
- `Company` — name of current employer
- `company_TYPE` — Unicorn / MNC / Mid-size / Early-stage. Case variants present (some 'unicorn', some 'UNICORN').
- `Skills` — comma-separated list of skills, e.g. 'Python, SQL, AWS, ML'. Some rows are NULL.
- `Location` — Bengaluru neighbourhood (Whitefield, ORR, Indiranagar, etc.)
- `Education_Tier` — Tier 1 / Tier 2 / Tier 3. Variants: 'Tier 1', 'T1', '1', 'Tier-1'.
- `Joining_Year` — year they joined current company
- `Work_Mode` — Remote / Hybrid / Onsite. Variants present.

IMPORTANT

Two warnings before you start.

1. The 'patterns' are real. The salary numbers are not random - they follow a formula combining role, experience, company type, education, and skill premiums. Your job is to surface these patterns. If you don't find them, your code has a bug.
2. ChatGPT cannot give you the answers. It can write the syntax for groupby. It cannot tell you which industry pays the most or which skill commands the highest premium - because it doesn't have your data. Every numerical insight in your final report must come from running the code, not from guessing.

Your 2 Hours - Six Tasks

Time budgets are guidelines. If you fall behind on one, do not skip the next - partial work on all six tasks scores better than perfect work on three. Track time on a timer.

TASK 1 · Load and inspect

Goal: Read the CSV. Inspect with `.info()`, `.shape`, `.dtypes`, `.isnull().sum()`, `.duplicated().sum()`. Print a short summary of what you found — column count, total rows, total duplicates, columns with the most missing data.

Concepts: `pd.read_csv`, `.info`, `.describe`, `.isnull`, `.duplicated`

Output: Print a 5-7 line summary of dataset quality. No code in the printed summary, just facts.

TASK 2 · Clean

Goal: Produce a clean DataFrame ready for analysis. Rename columns to snake_case. Standardise role variants to canonical names (write your own mapping dict). Convert CTC strings to floats (in LPA). Standardise education tier and company type. Drop duplicates. Handle NaN sensibly.

Specific traps to handle: `Current_CTC` has 4 string formats including '15,50,000' which is 15.5 LPA but looks like 1.5 million. Don't store it as 1550000 - convert correctly. `Previous_CTC` is NaN for freshers; do not fill these with 0 (would skew growth calculations).

Concepts: `.rename`, `.str.strip`, `.str.replace`, `pd.to_numeric(errors='coerce')`, dict-based `.replace`, `.drop_duplicates`, `.apply` with `lambda`

Output: Print `df.dtypes` after cleaning, plus the `value_counts` of `role_clean`, `company_type_clean`, `education_tier_clean` to verify.

TASK 3 · Five business questions

Goal: Answer the five questions below using the cleaned df. Print results clearly. Each question must end with a one-line insight written by YOU in a markdown cell explaining what the data says.

Time per question: 10-15 minutes each.

Comment requirement: Above each question's code block, write a comment explaining your approach. Below each result, write a markdown cell with the data-specific insight (not a generic statement).

The five questions (Task 3)

Q3.1 — CTC distribution by role. For every role, compute median, mean, min, and max CTC. Sort roles by median descending. Which role pays the most? Which the least? Is the mean materially different from the median in any role (sign of outliers)?

Q3.2 — Experience curve for SDE Backend. Bin SDE Backend professionals by years_exp into 4 buckets (0-1, 2-3, 4-5, 6+). Compute median CTC per bucket. What is the CTC growth per experience band? Hint: `pd.cut + groupby`.

Q3.3 — Skill premium for SDEs. Pick 4 skills from {AWS, ML, System Design, Kubernetes, Deep Learning, Microservices}. For each, compare median CTC of SDEs (any of Backend / Frontend / Full-Stack) who have the skill vs those who don't. Which skill adds the most to median CTC? Hint: `.str.contains` on the cleaned skills column.

Q3.4 — Company-type premium, same role. Pick one role (SDE Backend or Data Scientist). Compute median CTC across Unicorn / MNC / Mid-size / Early-stage for that role. What is the percentage premium of Unicorn over MNC at the same role?

Q3.5 — Underpaid professionals (multi-step). For every (role_clean, company_type_clean, exp_band) combination, compute median CTC. Then for every individual, compute the gap = `their_ctc - their_group_median`. Print the top 10 most-underpaid professionals with name, role, company_type, exp, ctc, group_median, gap. Filter to groups with at least 10 members (so the median is reliable). Hint: `groupby + transform + filter`.

TASK 4 · Build the printed report

Goal: Combine all findings into one final printed report. This is the LinkedIn screenshot.

Format: ASCII-art style with section dividers (= and -), aligned columns, percentages and absolute numbers. Treat formatting as a feature, not an afterthought.

Concepts: f-strings with width specifiers ('{name:<15}'), '=' * 60 for dividers, careful alignment

Target: Match the format on the next page. Numbers will be different — yours come from your analysis.

TASK 5 · Write three insights - in your own words

Goal: After the printed report, add a markdown cell with EXACTLY three insights. Each insight must:

- 1. Be data-specific.** Not 'Unicorns pay more' (generic). Yes: 'Unicorns pay 33% more than MNCs for SDE Backend at the same experience.'
- 2. Cite a number.** If your insight has no percentage or rupee value, it's not concrete enough.
- 3. Imply an action.** Each insight should suggest what a job-seeking student SHOULD do with this finding.

TASK 6 · Code review pass

Goal: Before submitting, do a fresh-eyes pass on your notebook.

Checklist: Every section starts with a markdown title. Every code cell has at least one comment explaining intent. Variable names are not 'x', 'a', 'temp'. The notebook runs top to bottom without errors. The final report cell is the LAST cell.

AI-assistance disclosure: If you used ChatGPT for any cell, add a comment: '# AI-assisted: <one line describing what you asked it'. Honesty is rewarded, not penalised.

The Target Printed Report

This is the shape your final printed output should take. Numbers shown here are illustrative — yours will be slightly different based on your tokenisation and filtering choices, but the structure and the patterns must match.

```
=====
BANGALORE TECH SALARY DECODER
Built by [Your Name] | The Unlox Academy | 2-Hour Live Project
=====

Dataset : 1,000 Bengaluru tech professionals (synthetic)
Period  : 2024 employment snapshot

---- MEDIAN CTC BY ROLE (in LPA) ----
Product Manager      ██████████ XX.X
Data Scientist       ██████████ XX.X
SDE Full-Stack       ██████████ XX.X
DevOps Engineer      ██████████ XX.X
SDE Backend          ██████████ XX.X
Business Analyst     ██████████ XX.X
UI/UX Designer       ██████████ XX.X
Data Analyst         ██████████ XX.X

---- SDE BACKEND CTC BY EXPERIENCE BAND ----
0 to 1 years   : XX.X LPA median
2 to 3 years   : XX.X LPA median (+XX% growth)
4 to 5 years   : XX.X LPA median (+XX% growth)
6+ years       : XX.X LPA median (+XX% growth)

---- SKILL PREMIUM FOR SDEs (median CTC) ----
Skill           With Skill Without Premium
AWS              XX.X LPA   XX.X   +XX%
ML               XX.X LPA   XX.X   +XX%
System Design    XX.X LPA   XX.X   +XX%
Kubernetes       XX.X LPA   XX.X   +XX%

---- COMPANY-TYPE PREMIUM (SDE Backend, same role) ----
Unicorn          XX.X LPA   <- baseline ceiling
MNC              XX.X LPA   (-XX% vs Unicorn)
Mid-size         XX.X LPA   (-XX% vs Unicorn)
Early-stage      XX.X LPA   (-XX% vs Unicorn)

---- TOP 5 MOST-UNDERPAID PROFESSIONALS ----
BLR0042  SDE Backend  MNC    3 yrs  gap: -X.X LPA
BLR0119  Data Analyst MNC    2 yrs  gap: -X.X LPA
...
=====
```

KEY INSIGHTS

1. <Your data-specific insight #1 with a number>
2. <Your data-specific insight #2 with a number>
3. <Your data-specific insight #3 with a number>

=====

Using ChatGPT Honestly - The Anti-LLM Section

We know you will use ChatGPT, Claude, Gemini, or similar tools during this 2-hour project. We are not banning them. We are also not pretending they cannot help. They can. But used badly, they will sink you in this specific project. Here is the honest guidance.

What LLMs are good for (use them for this)

- Reminding you of pandas syntax - 'how do I do a groupby with multiple columns again?'
- Debugging an error message you don't understand
- Suggesting a structure for the printed output
- Reviewing your code for bugs after you've written it

What LLMs are useless for (do not rely on them)

- Telling you which role pays the most — they don't have your data, they're guessing or hallucinating numbers
- Telling you the 'right' role-name standardisation mapping — only your data has the specific variants. You must inspect first.
- Writing your 3 final insights — these must come from running your code. Any LLM-generated insight will be generic and Girish will recognise it instantly.

IMPORTANT

The four traps that ChatGPT will likely fail.

Trap 1: Role standardisation. ChatGPT will give you a generic mapping like {'Backend Dev': 'Software Engineer'}. Your dataset has SPECIFIC variants ('SDE Backend', 'BE', 'Backend Engineer', 'Backend Developer', 'Backend Dev') that all map to ONE canonical name. INSPECT `df['Role'].unique()` first. Then build the dict.

Trap 2: CTC parsing. ChatGPT will give you `pd.to_numeric`. But '15,50,000' is 15.5 LPA, not 1.5 million. Your parser must detect the 'LPA' suffix or check the magnitude.

Trap 3: Underpaid computation. ChatGPT will tell you to compute median and filter. But Task 3.5 requires `groupby + transform + filter` for group size. A naive approach gives wrong answers for small groups.

Trap 4: Final insights. ChatGPT will write something like 'Bangalore tech salaries vary by experience.' This will get you zero. The required format: a specific number, a specific comparison, an actionable next step.

If we see three insights that look like ChatGPT generic outputs, the entire Insights section is marked zero. Be honest with yourself about whether your insight could have been written by someone who hadn't seen the data.

Code Comments Expected

Comments matter as much as code in this project. A clean notebook with sparse commenting scores below a moderately-buggy notebook with thoughtful comments. Here's what to comment, and where.

Cell-level comments (top of every code cell)

One short comment explaining what THIS cell does and why. Example:

```
# Clean Current_CTC: 4 formats present – '15.5 LPA', '15.5', '15,50,000', '₹15.5 LPA'.  
# Strategy: strip non-numeric chars, parse as float, divide by 100000 if it looks like rupees.  
def parse_ctc(s):  
    ...
```

Line-level comments (where logic is non-obvious)

Inside a cell, comment any line that takes a non-obvious decision. Don't comment obvious lines.

```
df['ctc_lpa'] = df['Current_CTC'].apply(parse_ctc)    # parsed to float, in LPA units  
df = df.dropna(subset=['ctc_lpa'])    # drop rows where CTC couldn't be parsed
```

Markdown cell after every analysis question

After answering each Q3.1 through Q3.5, write a 1-2 line markdown cell with the insight. Format:

```
### Insight (Q3.1)  
Product Managers earn the highest median CTC at XX.X LPA,  
which is X.Xx the median of Data Analysts (lowest at XX.X LPA).  
This is the biggest cross-role gap in the dataset.
```

AI-assistance disclosure

If you copy-pasted ChatGPT code into any cell, add this comment at the top of that cell:

```
# AI-assisted: asked ChatGPT for the pd.cut syntax with custom bin edges
```

Honest disclosure earns you marks. Silent copy-paste, when detected, earns you zero on that section.

Submission and LinkedIn Sharing

Submission (the academic deliverable)

- One .ipynb file named: LiveProject_Salary_Decoder_<YourName>.ipynb
- Upload to Google Colab or GitHub. Make the share setting 'Anyone with link can view'.
- Submit only the LINK via the Google Form Girish shares in the cohort Slack.
- Deadline: end of the 2-hour session.

LinkedIn post (the career deliverable)

Within 24 hours of completing the project, post on LinkedIn. Template you can adapt:

I spent 2 hours today analyzing 1,000 Bangalore tech salaries (synthetic dataset designed by my academy).

Three findings that surprised me:

1. <Your data-specific insight #1>
2. <Your data-specific insight #2>
3. <Your data-specific insight #3>

Built using only Python fundamentals, Pandas, and NumPy.
No ML. No external APIs. Just clean analytics on messy data – the kind of work an HR analytics team does every day.

Notebook: <Colab link>

#DataAnalytics #Python #Pandas #CareerInTech
#UnloxAcademy

Tag The Unlox Academy in your post. Pin it to your profile. This single post will outperform any internship listing you currently have on your resume.

Evaluation Rubric - 100 marks

- Task 1 — Load and inspect: 5 marks. Did they correctly identify dataset shape, dtypes, missing counts, duplicates?
- Task 2 — Cleaning: 25 marks. (Role standardisation 8, CTC parsing 8, company_type standardisation 4, education tier 3, duplicates 2.)
- Task 3 — Five business questions: 35 marks (7 each). Each question scored on: correct technique (3), correct answer (3), insight quality (1).
- Task 4 — Printed report: 15 marks. Formatting, alignment, completeness of all 5 sections.
- Task 5 — Three insights: 10 marks. Data-specific (4), number-backed (3), action-implied (3). Generic LLM-style insights get zero.
- Task 6 — Code review and comments: 10 marks. Comment density, variable naming, notebook runs top-to-bottom, AI disclosure honesty.

Pass mark: 60. Distinction: 80+. Below 60 = re-attempt in next batch.

IMPORTANT

Red flags that earn automatic deductions.

Notebook does not run top-to-bottom: -15 marks.

Used pandas methods not yet taught (read_excel, pivot fancy, plotly): -5 each.

Used regex when the lesson said don't: -5 per cell.

Insights look ChatGPT-generic: -10 (entire insights section zero).

Three or more cells with no comments at all: -5.

Silent AI copy-paste (no disclosure) detected: -15 on that section.